PROBLEM. Property (contributed by N. R. Lim-Cheng)

Owning land is one of the goals a person has. Before buying land, the prospective buyer would normally have a size in mind already, from number of rooms he wants to have in his future home, including the size of the rooms. Due to urban planning, land sellers impose certain restrictions, maybe depending on the area (like maximum number of floors that can be built and even amount of open space, so homes are not all attached to each other.

Consider the following diagram:

The bigger rectangle shows a visualization of the entire property area, the shaded small rectangle is the buildable area, and the unshaded portion is the open space.

For this problem, your task is to implement five functions. Details are as follows:

(1) Compute for the buildable area [10 pts]
- Implement function getBuildArea() that will accept as parameter the property area, in square meters (m^2), and the percentage of buildable space.

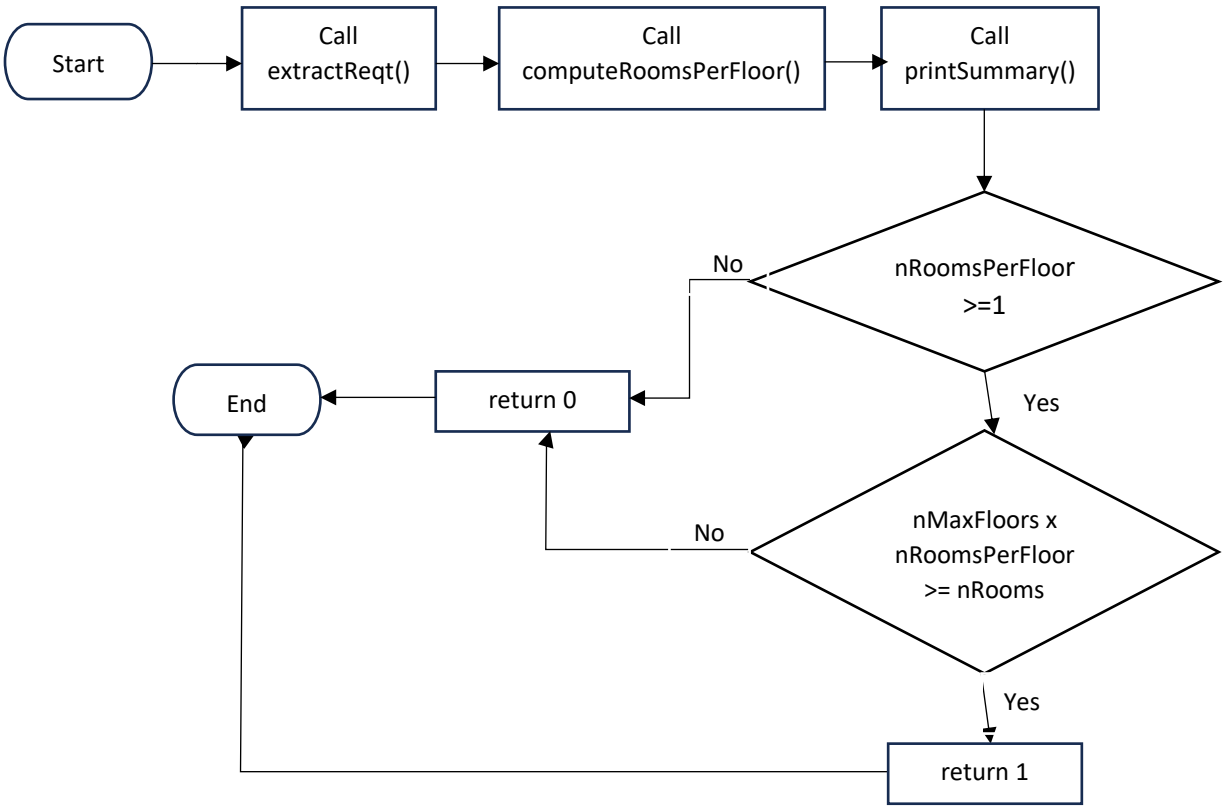
(2) Compute the number of rooms. [5 pts]
- Implement function computeRoomsPerFloor() that will accept as parameter the minimum size of each room (in square meters) and the buildable space (in square meters). This latter is the value computed from item (1) requirement.

(3) Extract the buyer's requirement [10 pts]
- Implement function extractReqt() that will accept the requirement specifications of the user represented as a 5 digit positive integer. Whether the buyer wants a high ceiling or not, the number of rooms, and minimum size in square meters per room are updated via pointers received in the function.
- The given integer is in the format: RCSSS
  - R - total rooms, assume to be from 1 to 9 only. This includes count for living room, bathroom, kitchen, etc. This does not count space needed when high ceiling is needed
  - C - 1 for high ceiling, 0 for regular ceiling
  - SSS - the rightmost 3 digits refer to the minimum size of each room, in square meters
- Note that implementation of this function should NOT have any conditional statements. Violation of this restriction will invalidate any score you get from this.

(4) Complete the printSummary() function [10 pts]
- Look at results of this function after the text Summary in the sample runs below. Note that output format should be exactly the same, including spacing and text displayed (meaning, there should be no additional or less text, space, or values displayed, if the same inputs are used to test your program). Note that all strings occupy 18 spaces, all integers occupy 5 spaces, and all floats occupy 8 spaces including the decimal point and 2 decimal places.

(5) Determine if the property being sold falls within the requirements of the buyer [15 pts]

- Implement function okToBuy() that will return 1 if the buyer's requirement is met by the property being sold, and returns 0 otherwise. For those examples listed in item 4 below, where last displayed output indicate "NOT met", it means okToBuy() returned 0.
- Note that implementation of this function should NOT have any conditional statements. Violation of this restriction will invalidate any score you get from this.
- Hints: relational and/or logical expression in return statement.
- The implementation of this function should follow the algorithm from the given flowchart:



If codes from lines 45 to 60 in propertyMain.c are commented out, no inputs are asked and the following are the generated result:

```
Testing getBuildSize() = 4656.71
Testing computeRoomsPerFloor() = 52
Testing extractReqt():
R = 9; C = 0; SSS = 801
Testing printSummary():
Summary:
Build space       20638.90
Open space          111.22

Buyer Needs:
      High Ceiling    1
             Rooms    3
              Size   45
```

Example Runs below are from commenting out lines 23 t0 41 in propertyMain.c

Example Run 1:

```
Enter property size for sale: 315.75
Enter percentage of property that can be built on: 75
Enter maximum floors for this property: 2
Enter requirement of buyer: 31020

Summary:
Build space         236.81
Open space           78.94

Buyer Needs:
      High Ceiling    1
             Rooms    3
              Size   20

Buyer requirement is met by this property.
```

Example Run 2:

```
Enter property size for sale: 123
Enter percentage of property that can be built on: 50
Enter maximum floors for this property: 1
Enter requirement of buyer: 21030

Summary:
Build space          61.50
Open space           61.50

Buyer Needs:
      High Ceiling    1
             Rooms    2
              Size   30

Buyer requirement is met by this property.
```

Example Run 3: not met because 1 room already cannot fit in 61.50 sqm (lot area = area per floor)

```
Enter property size for sale: 123
Enter percentage of property that can be built on: 50
Enter maximum floors for this property: 2
Enter requirement of buyer: 20075

Summary:
Build space          61.50
Open space           61.50

Buyer Needs:
      High Ceiling    0
             Rooms    2
              Size   75

Buyer requirement is NOT met by this property.
```

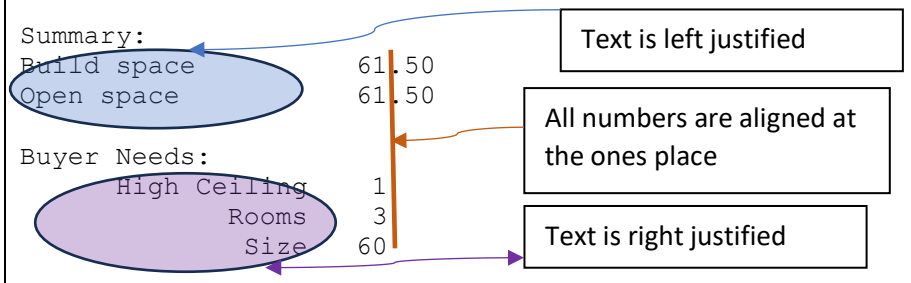
Example Run 4: though each room can fit in 61.50 sqm, buyer needs at least 3 rooms of that size.

```
Enter property size for sale: 123
Enter percentage of property that can be built on: 50
Enter maximum floors for this property: 2
Enter requirement of buyer: 31060

Summary:
Build space          61.50
Open space           61.50

Buyer Needs:
      High Ceiling    1
             Rooms    3
              Size   60

Buyer requirement is NOT met by this property.
```



You are given three (3) files for this problem:

(1) **`property.h`** which contains the function prototypes that will be used in the program. This file cannot be modified.

(2) **`Property-LASTNAME.c`** which contains some initial code that you'll need to complete. Comments indicate the requirement and restrictions imposed for the solution. For this file, the student's implementation should not include any scanf(). Only the function printSummary() can have the printf(). This file should not be modified to contain the main(). Other requirements and restrictions are indicated in the file.

(3) **`propertyMain.c`** which contains the main() to produce the sample runs indicated.

To debug and test your program using a user-defined module (.h) and with the main() in a separate file, you need to do the following:

1. You need to have all three files stated above in the same folder (directory).
2. Then, use the following instruction to compile: **`gcc -Wall -std=c99 propertyMain.c`**
   This will produce a.exe. Note that there is no need to compile Property-LASTNAME.c separately.
3. Run in the command prompt as usual. For example, given a.exe generated from step 2, just type **`a`** in the command prompt.

DELIVERABLES: Your C program source file **`Property-LASTNAME.c`** with your own last name as filename. For example, if your lastname is SANTOS, then the source file should be named as `Property-SANTOS.c`. Upload your source file in AnimoSpace before the indicated submission time.

TESTING & SCORING:

- Your program will be compiled via gcc -Wall -std=c99. Thus, for each function that does not compile successfully, the score for that function is 0.
- Your program will be tested by your instructor with other sample inputs (different from the ones given to you) and with function calls of different parameter values. It is also possible for the instructor to test your property.c file (the functions) using a different main() program.
- Full credit will be given for the function only if the student's implementation are all correct for all the test values used by the instructor during checking AND only if the student's implementation complied with the requirement and did not violate restrictions. Deductions will be given if not all test cases produce correct results OR if restrictions were not followed.